Name: Ishaan Vijaykumar Dhore

PRN: 126224310

Roll no.: 29

Panel: G

School of Computer Science and Engineering
Department of Computer Engineering and Technology

S. Y. B. Tech CSE
AY 2026-27 Semester 3

Computer Networks Lab (CSE20410)
CN Lab Assignment: 6

Title: Write a program UDP Socket Programming: (Language: C/C++/Python/Java).

Aim: Write a C program for wired network using UDP socket to perform any one of the following operations

a. String Conversion from UpperCase to LowerCase (50% students)
b. Reverse the String (Remaining 50% students)

Objectives:

1. To understand concept of socket programming using UDP.

Theory:

1. UDP Header

UDP (User Datagram Protocol) uses a very simple, fixed 8-byte header, which is much smaller than the TCP header since UDP does not maintain connection state, acknowledgements, or sequencing. The header consists of four fields, each 16 bits (2 bytes) long, as shown below.

Field	Size	Description
Source Port	16 bits	Port number of the sending application. Optional — set to 0 if unused.
Destination Port	16 bits	Port number of the receiving application on the destination host.
Length	16 bits	Total length of the UDP header plus the data, in bytes.
Checksum	16 bits	Used for error-checking of the header and data; optional in IPv4.

Because the header carries no sequence numbers or acknowledgement fields, UDP adds almost no overhead to the data being sent — which is exactly why it is preferred for applications where speed matters more than guaranteed delivery.

2. UDP Socket Primitives at Server and Client

Unlike TCP, UDP socket programming does not involve establishing a connection before data is exchanged — every datagram is sent independently, carrying the destination address with it.

On the server side, the primitives are used in the following order:

- socket() — creates a new UDP socket and returns a socket descriptor.
- bind() — associates the socket with a specific IP address and port number on the local machine.
- recvfrom() — waits for and receives a datagram from any client, along with the sender's address.
- sendto() — sends a reply datagram back to the client's address.
- close() — terminates the socket and releases the associated resources.

On the client side, the primitives are used in the following order:

- socket() — creates a new UDP socket on the client machine.
- sendto() — sends a datagram directly to the server's IP address and port number, with no prior connection setup.
- recvfrom() — receives the reply datagram from the server.
- close() — terminates the socket once communication is complete.

Since there is no listen() or accept() call, and no connect() call on the client either, the server can receive datagrams from any client that sends to its bound port — this is what makes UDP connectionless.

Pseudocode:

Server Program:

2. START
3. Create UDP socket
4. Bind socket to IP address and port number
5. Receive string from client
6. Convert received string to lowercase
7. Send converted string back to client
8. Close socket
9. STOP

Client Program:

10. START
11. Create UDP socket
12. Input string from user
13. Send string to server
14. Receive converted string from server
15. Display converted string
16. Close socket
17. STOP

Program Code:

Server Program

```
#include <iostream>
#include <string>
#include <cctype>
#include <winsock2.h>

int main() {
    WSADATA wsa;
    WSAStartup(MAKEWORD(2, 2), &wsa);

    SOCKET server_socket = socket(AF_INET, SOCK_DGRAM, 0);

    sockaddr_in server_addr = {AF_INET, htons(8080)};
    server_addr.sin_addr.s_addr = INADDR_ANY;
    bind(server_socket, (sockaddr*)&server_addr, sizeof(server_addr));

    std::cout << "UDP Server waiting for messages...\n";

    char buffer[1024];
    sockaddr_in client_addr;
    int client_len = sizeof(client_addr);

    while (true) {
        memset(buffer, 0, sizeof(buffer));

        recvfrom(server_socket, buffer, 1024, 0, (sockaddr*)&client_addr, &client_len);
        std::string msg(buffer);
        std::cout << "Received from client: " << msg << "\n";

        for (char &c : msg) {
            c = std::tolower(c);
        }

        sendto(server_socket, msg.c_str(), msg.length(), 0, (sockaddr*)&client_addr, client_len);
        std::cout << "Converted and sent back: " << msg << "\n\n";
    }

    closesocket(server_socket);
    WSACleanup();
    return 0;
}
```

Client Program

```
#include <iostream>
#include <string>
#include <winsock2.h>

int main() {
    WSADATA wsa;
    WSAStartup(MAKEWORD(2, 2), &wsa);

    SOCKET client_socket = socket(AF_INET, SOCK_DGRAM, 0);

    sockaddr_in server_addr = {AF_INET, htons(8080)};
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    int server_len = sizeof(server_addr);

    std::string msg;
    std::cout << "Enter an UPPERCASE string to send: ";
    std::getline(std::cin, msg);

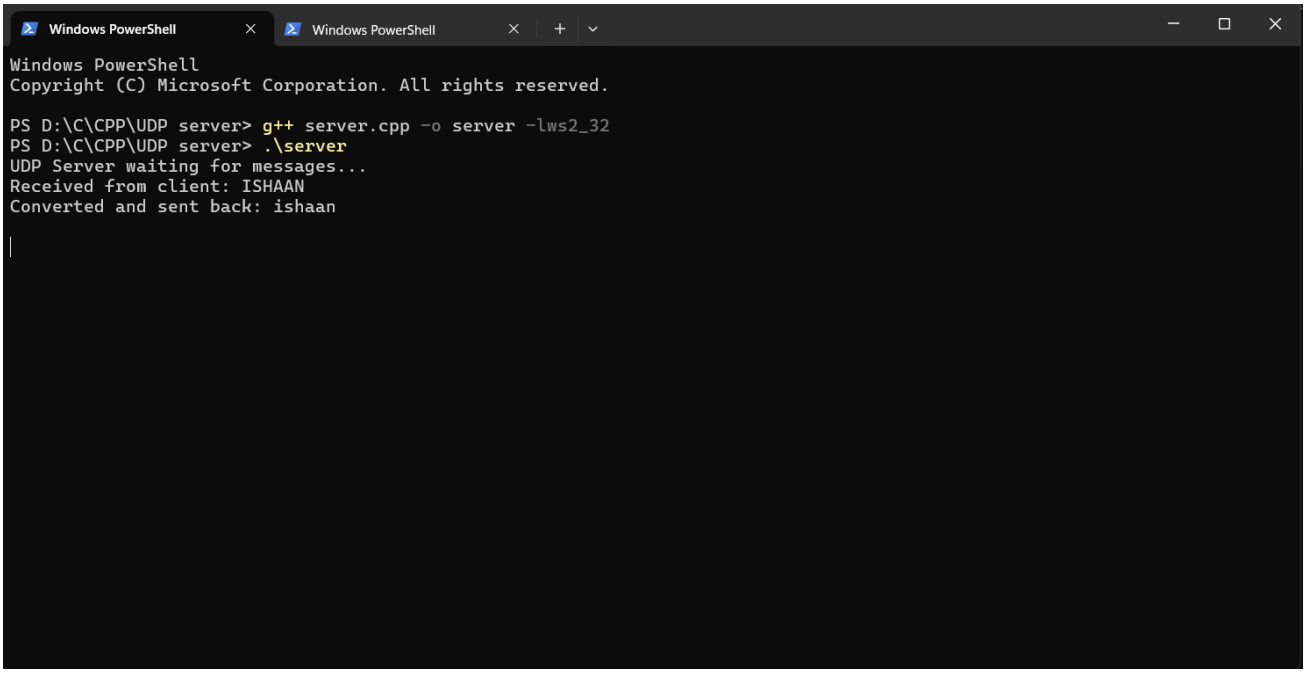
    sendto(client_socket, msg.c_str(), msg.length(), 0, (sockaddr*)&server_addr, sizeof(server_addr));

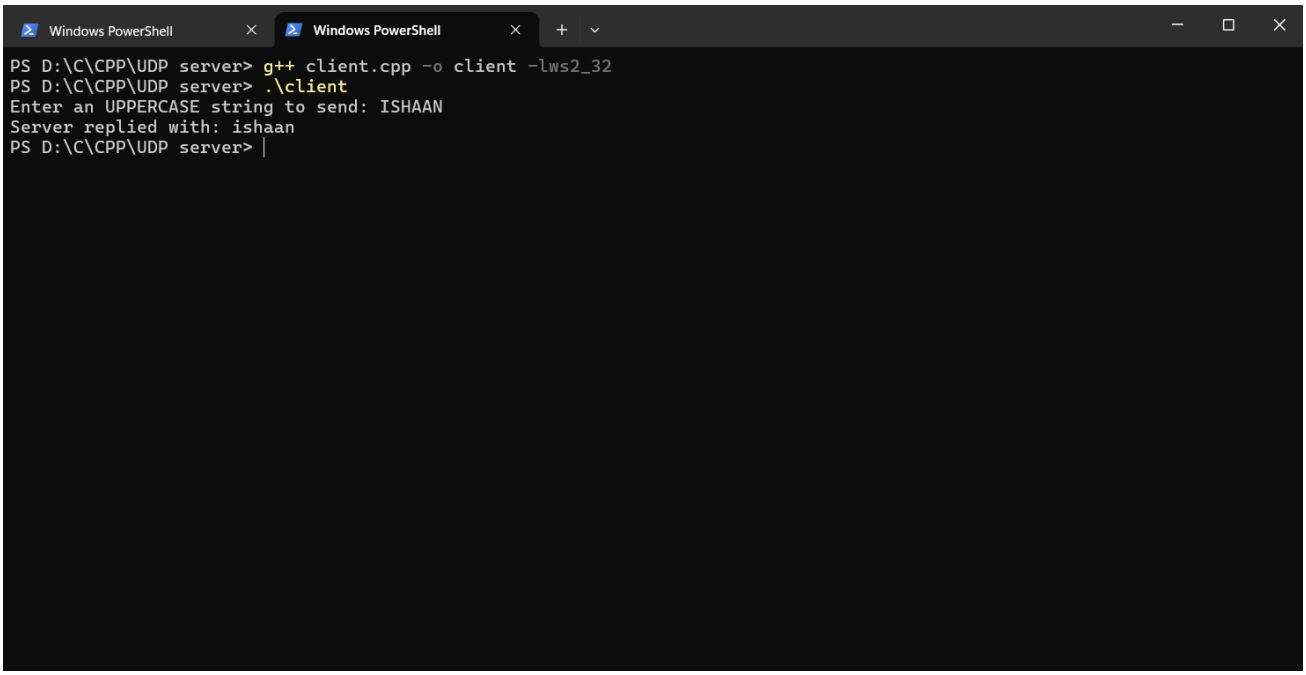
    char buffer[1024] = {0};
    recvfrom(client_socket, buffer, 1024, 0, (sockaddr*)&server_addr, &server_len);

    std::cout << "Server replied with: " << buffer << "\n";

    closesocket(client_socket);
    WSACleanup();
    return 0;
}
```

Output Screenshots:





Students Observation:

Running this lab made the difference between TCP and UDP much more tangible than just reading about it. Since there was no connection setup involved, the client could directly send its string to the server using sendto() without first going through a connect() step, and the server simply waited on recvfrom() for any incoming datagram. The uppercase string entered on the client was correctly received and converted to lowercase by the server, and the reply came back and displayed properly on the client side. Because UDP doesn't guarantee delivery or ordering, I understood why it's suited to simple, fast request-reply exchanges like this one, where retransmission logic would be unnecessary overhead. Overall, the exercise helped connect the theory of UDP being lightweight and connectionless to an actual working client-server program built using sendto()/recvfrom().

FAQ:

1) What is UDP and why is it used in socket programming?

UDP (User Datagram Protocol) is a connectionless, lightweight transport layer protocol that sends data in independent units called datagrams, without establishing a connection beforehand and without guaranteeing delivery, ordering, or duplicate protection. It is used in socket programming whenever speed and low overhead matter more than reliability, since it avoids the delays of connection setup, acknowledgements, and retransmissions that TCP requires. This makes UDP well suited to applications like DNS lookups, live video/audio streaming, and online gaming, where occasional lost packets are acceptable but delay is not.

2) What are the important functions used in UDP socket programming?

The core functions used in UDP socket programming are socket(), which creates the UDP socket (using SOCK_DGRAM); bind(), which assigns a specific IP address and port number to the socket (typically done on the server); sendto(), which sends a datagram to a specified destination address without needing a prior connection; recvfrom(), which receives a datagram and also returns the address of the sender, which is essential since no connection is maintained; and close(), which releases the socket once communication is complete.

3) What is the difference between TCP and UDP socket programming?

Parameter	TCP Socket Programming	UDP Socket Programming
Connection	Connection-oriented: requires connect() (client) and listen()/accept() (server) before data transfer.	Connectionless: no connection setup; sendto()/recvfrom() used directly.
Reliability	Reliable — guarantees delivery, ordering, and error-checking via acknowledgements and retransmission.	Unreliable — no guarantee of delivery, ordering, or duplicate protection.
Key primitives	socket(), bind(), listen(), accept(), connect(), send(), recv(), close()	socket(), bind(), sendto(), recvfrom(), close()
Header size	20 bytes (minimum)	8 bytes
Speed	Slower, due to connection setup and acknowledgements.	Faster, since there is no connection setup or acknowledgement overhead.
Typical use	File transfer, web browsing, email — where accuracy matters.	DNS queries, video/audio streaming, online gaming — where speed matters more than accuracy.

4) What happens if a UDP packet is lost during transmission?

If a UDP datagram is lost in transit, UDP itself does nothing to detect or recover it — there is no acknowledgement mechanism, sequence numbering, or automatic retransmission built into the protocol, so the packet is simply gone. It is entirely up to the application layer to notice the loss (for example, by using timeouts or expecting a reply that never arrives) and to decide whether to resend the data or simply continue without it.

5) State five applications of UDP and What is Ephemeral Ports?

Five common applications of UDP are DNS (Domain Name System) queries, which need quick request-reply lookups; video and audio streaming services, where a few dropped packets are preferable to delay; VoIP (Voice over IP) calls; online multiplayer gaming, which needs fast, real-time updates; and DHCP (Dynamic Host Configuration Protocol), used for assigning IP addresses on a network.

Ephemeral ports are temporary port numbers automatically assigned by the operating system to the client side of a connection, typically from a range above 1024 (commonly 49152–65535), so that the client doesn't need to pick or reserve a fixed port itself. When a client creates a socket and sends data without explicitly calling bind(), the OS assigns it one of these short-lived ports, which is used only for the duration of that communication and released afterward.